

1 Carbon-Aware Scheduling: MILP Formulation

1.1 Problem Definition

We formulate the carbon-aware scheduling problem as a Mixed Integer Linear Programming (MILP) problem. The objective is to find an optimal assignment of pods (computational tasks) to nodes (computing resources) and timeslots while minimizing the total carbon emissions.

1.2 Sets and Indices

$$i \in \mathcal{P} \quad \text{Set of pods} \quad (1)$$

$$j \in \mathcal{F} \quad \text{Set of nodes} \quad (2)$$

$$t \in \mathcal{T} \quad \text{Set of timeslots} \quad (3)$$

1.3 Parameters

$D_i \in \mathbb{N}^+$	Deadline timeslot for pod i	(4)
$s_i \in \mathbb{N}^+$	Earliest timeslot for pod i	(5)
$d_i \in \mathbb{N}^+$	Duration of pod i (in timeslots)	(6)
$w_i \in \mathbb{N}^+$	Temporal slack for pod i	(7)
$T_{max} \in \mathbb{N}^+$	Maximum available timeslot index	(8)
$c_i \in \mathbb{R}^+$	CPU request of pod i (cores)	(9)
$r_i \in \mathbb{R}^+$	RAM request of pod i (MB)	(10)
$C_{jt} \in \mathbb{R}^+$	Available CPU of node j in timeslot t (cores)	(11)
$R_{jt} \in \mathbb{R}^+$	Available RAM of node j in timeslot t (MB)	(12)
$CI_{j,t} \in \mathbb{R}^+$	Carbon intensity of node j in timeslot t (g/kWh)	(13)
$P_j^{\text{idle}}, P_j^{\text{active}}, P_j^{\text{max}} \in \mathbb{R}^+$	Idle, active, and max power of node j (W)	(14)
$\text{CPU}_j^{\text{tot}} \in \mathbb{R}^+$	Total CPU capacity of node j (cores)	(15)
$\text{EC}_j \in \mathbb{R}^+$	Embodied carbon of node j (g CO ₂ e)	(16)
$L_j \in \mathbb{R}^+$	Lifetime of node j (hours)	(17)

The carbon emissions e_{ijt} are calculated considering:

- Operational carbon: Based on the pod's resource consumption, node's power profile, and regional carbon intensity forecasts across the pod's execution duration
- Embodied carbon: Accounting for the amortized manufacturing emissions of the hardware over its lifetime

1.4 Decision Variables

$$x_{ijt} = \begin{cases} 1 & \text{if pod } i \text{ is placed on node } j \text{ starting at timeslot } t \\ 0 & \text{otherwise} \end{cases} \quad (18)$$

$$s_i \in \{0, 1\} \quad \text{unplaced indicator for pod } i \text{ (1 if unplaced)} \quad (19)$$

$$y_{jt} \in \{0, 1\} \quad \text{activation of node } j \text{ in timeslot } t \quad (20)$$

1.5 Feasible Placements

Let $\mathcal{V}_i \subset \mathcal{F} \times \mathcal{T}$ be the set of valid node-timeslot pairs for pod i :

$$\mathcal{V}_i = \{(j, t) \in \mathcal{F} \times \mathcal{T} \mid t \geq s_i \text{ and} \quad (21)$$

$$t + d_i \leq D_i \text{ and} \quad (22)$$

$$C_{j(t+k)} \geq c_i \text{ for all } k \in \{0, 1, \dots, d_i - 1\} \text{ and} \quad (23)$$

$$R_{j(t+k)} \geq r_i \text{ for all } k \in \{0, 1, \dots, d_i - 1\}\} \quad (24)$$

The deadline timeslot D_i for pod i is calculated as:

$$D_i = s_i + d_i + w_i \leq T_{max} \quad (25)$$

where the deadline includes the pod's execution duration and temporal slack for scheduling flexibility.

1.6 Mathematical Model

1.6.1 Lexicographic Objective

We solve the problem in two phases.

Phase 1 (Placement Feasibility).

$$\text{Minimize } Z_1 = \sum_{i \in \mathcal{P}} s_i \quad (26)$$

$$\text{s.t. } \sum_{(j,t) \in \mathcal{V}_i} x_{ijt} + s_i = 1 \quad \forall i \in \mathcal{P} \quad (27)$$

$$\text{Capacity constraints (CPU/RAM) as in (31)-(32)} \quad (28)$$

Phase 2 (Emissions Minimization). Fix the set of unplaced pods to the Phase 1 optimum (i.e., each s_i is fixed to its Phase 1 value) and minimize total

emissions:

$$\text{Minimize } Z_2 = \sum_{i \in \mathcal{P}} \sum_{(j,t) \in \mathcal{V}_i} \left(\sum_{k=0}^{d_i-1} \text{CI}_{j,t+k} \frac{(P_j^{\max} - P_j^{\text{active}})}{1000} \frac{c_i}{\text{CPU}_j^{\text{tot}}} \right) x_{ijt} \quad (29)$$

$$+ \sum_{j \in \mathcal{F}} \sum_{t \in \mathcal{T}} \left(\text{CI}_{j,t} \frac{P_j^{\text{idle}}}{1000} + \frac{\text{EC}_j}{L_j} \right) y_{jt} \quad (30)$$

subject to the Phase 1 assignment equations, fixed s_i , capacity constraints, and activation linking (big- M):

$$\sum_{i \in \mathcal{P}} \sum_{t' \in \mathcal{T}_{it}} c_i x_{ijt'} \leq C_{jt} \quad \forall j, t \quad (31)$$

$$\sum_{i \in \mathcal{P}} \sum_{t' \in \mathcal{T}_{it}} r_i x_{ijt'} \leq R_{jt} \quad \forall j, t \quad (32)$$

$$\sum_{i \in \mathcal{P}} \sum_{t' \in \mathcal{T}_{it}} c_i x_{ijt'} \leq M_{jt} y_{jt} \quad \forall j, t \quad (33)$$

where M_{jt} can be chosen as C_{jt} (or another valid upper bound on CPU usage).

1.6.2 Constraints

Assignment constraint (Phase 1 and Phase 2):

$$\sum_{(j,t) \in \mathcal{V}_i} x_{ijt} + s_i = 1 \quad \forall i \in \mathcal{P} \quad (34)$$

CPU capacity constraints: The total CPU usage in each node and timeslot must not exceed capacity:

$$\sum_{i \in \mathcal{P}} \sum_{t' \in \mathcal{T}_{it}} c_i \cdot x_{ijt'} \leq C_{jt} \quad \forall j \in \mathcal{F}, \forall t \in \mathcal{T} \quad (35)$$

where $\mathcal{T}_{it} = \{t' \in \mathcal{T} \mid t' \leq t < t' + d_i, (j, t') \in \mathcal{V}_i\}$ represents the set of starting timeslots t' such that pod i would be running at timeslot t if started at t' .

RAM capacity constraints: The total RAM usage in each node and timeslot must not exceed capacity:

$$\sum_{i \in \mathcal{P}} \sum_{t' \in \mathcal{T}_{it}} r_i \cdot x_{ijt'} \leq R_{jt} \quad \forall j \in \mathcal{F}, \forall t \in \mathcal{T} \quad (36)$$

Binary constraint: Decision variables are binary:

$$x_{ijt} \in \{0, 1\} \quad \forall i \in \mathcal{P}, \forall (j, t) \in \mathcal{V}_i \quad (37)$$

1.7 Emissions Calculation

The Phase 2 objective decomposes emissions into:

- **Dynamic (per pod):** For each placement (i, j, t) , and each offset $k \in \{0, \dots, d_i - 1\}$, the marginal power is $(P_j^{\max} - P_j^{\text{active}})(c_i/\text{CPU}_j^{\text{tot}})$ (W). Operational emissions (g) are $\text{CI}_{j,t+k} \frac{(P_j^{\max} - P_j^{\text{active}})}{1000} \frac{c_i}{\text{CPU}_j^{\text{tot}}}$.
- **Idle + embodied (per node,slot):** Paid once via y_{jt} with emissions $\text{CI}_{j,t} \frac{P_j^{\text{idle}}}{1000} + \frac{\text{EC}_j}{L_j}$.

Post-solve reporting can allocate the idle+embodied term proportionally to each pod's CPU share in an active slot.

1.8 Solver Fallback for Large Instances

For large instances where the full Phase 2 with activation variables times out, we employ a dynamic-only fallback: remove y_{jt} and the activation linking constraints, and minimize only the dynamic part of Z_2 . This preserves lexicographic feasibility (the exact unplaced set from Phase 1) while providing a tractable emissions minimization.

1.8.1 Theoretical Foundation

Let $\mathcal{M}$ denote the original MILP formulation. The feasibility of $\mathcal{M}$ depends on the existence of a solution $\mathbf{x} \in \{0, 1\}^{|\mathcal{P}| \times |\mathcal{F}| \times |\mathcal{T}|}$ satisfying constraints (14)-(17). When $\mathcal{M}$ is infeasible, we seek to solve a relaxed problem $\mathcal{M}'$ that:

1. Guarantees feasibility: $\mathcal{M}'$ always admits a solution
2. Maximizes pod placement: Priority is given to scheduling as many pods as possible
3. Minimizes carbon emissions: Among solutions with equal placement counts, prefer lower emissions
4. Maintains computational tractability: Relaxation overhead should be minimal

1.8.2 Multi-Objective Optimization with Penalty Functions

Our primary relaxation strategy transforms the hard assignment constraints into soft constraints using penalty functions. This approach, known as *goal programming* in the operations research literature, converts the infeasible MILP into a bi-objective optimization problem.

Extended Decision Variables: We introduce slack variables to capture unplaced pods:

$$y_i \in \{0, 1\} \quad \forall i \in \mathcal{P} \quad (38)$$

where $y_i = 1$ indicates that pod i remains unplaced.

Reformulated Objective Function: The bi-objective formulation becomes:

$$\text{Minimize } Z(\mathbf{x}, \mathbf{y}) = \lambda \sum_{i \in \mathcal{P}} y_i + \sum_{i \in \mathcal{P}} \sum_{(j,t) \in \mathcal{V}_i} e_{ijt} \cdot x_{ijt} \quad (39)$$

where $\lambda \gg \max_{i,j,t} e_{ijt}$ is a penalty weight ensuring lexicographic optimization priority.

Softened Assignment Constraints: The hard assignment constraint (14) is replaced with:

$$\sum_{(j,t) \in \mathcal{V}_i} x_{ijt} + y_i = 1 \quad \forall i \in \mathcal{P} \quad (40)$$

This constraint ensures that each pod is either placed exactly once ($\sum x_{ijt} = 1, y_i = 0$) or marked as unplaced ($\sum x_{ijt} = 0, y_i = 1$).

Theoretical Properties:

Theorem 1.1 (Feasibility Guarantee). *The relaxed MILP formulation $\mathcal{M}'$ with objective (20) and constraints (21), (15)-(17) is always feasible.*

Proof. The solution $\mathbf{x} = \mathbf{0}$ and $y_i = 1$ for all $i \in \mathcal{P}$ satisfies all constraints trivially, as no resources are consumed when all pods are unplaced. $\square$

Theorem 1.2 (Optimality Preservation). *For sufficiently large penalty λ , if the original problem $\mathcal{M}$ is feasible, then the optimal solution to $\mathcal{M}'$ places all pods (i.e., $\mathbf{y} = \mathbf{0}$) and achieves the same objective value as $\mathcal{M}$.*

Proof. If $\mathcal{M}$ is feasible with optimal value Z^* , then $\mathcal{M}'$ has a feasible solution with objective value Z^* . Any solution with unplaced pods has objective value at least $\lambda + Z^* > Z^*$ for $\lambda > 0$, thus the optimal solution to $\mathcal{M}'$ must place all pods. $\square$

1.8.3 Iterative Constraint Relaxation

When the multi-objective approach encounters solver limitations or remains computationally intractable, we apply an iterative constraint relaxation strategy that systematically reduces problem complexity.

Constraint Pressure Quantification: We define a constraint pressure metric ϕ_i for each pod i :

$$\phi_i = \alpha \cdot \frac{1}{|\mathcal{V}_i|} + \beta \cdot \max\left(0, \frac{\tau_{max} - \tau_i}{\tau_{max}}\right) \quad (41)$$

where:

- $|\mathcal{V}_i|$ is the number of feasible placements for pod i
- $\tau_i = D_i - s_i - d_i$ is the scheduling flexibility (slack) for pod i

- $\tau_{max} = 5$ is the maximum considered slack threshold
- $\alpha = 10$ and $\beta = 1$ are empirically determined weights

Iterative Relaxation Algorithm:

Algorithm 1 Iterative Constraint Relaxation

```

1: Input: Pod set  $\mathcal{P}$ , maximum iterations  $k_{max}$ , removal fraction  $\rho = 0.1$ 
2: Initialize:  $\mathcal{P}^{(0)} = \mathcal{P}$ ,  $\mathcal{P}^{removed} = \emptyset$ ,  $k = 0$ 
3: while  $|\mathcal{P}^{(k)}| > 0$  and  $k < k_{max}$  do
4:   Solve MILP  $\mathcal{M}(\mathcal{P}^{(k)})$  with pod set  $\mathcal{P}^{(k)}$ 
5:   if  $\mathcal{M}(\mathcal{P}^{(k)})$  is feasible then
6:     return optimal solution and  $\mathcal{P}^{removed}$ 
7:   end if
8:   Compute constraint pressure  $\phi_i$  for all  $i \in \mathcal{P}^{(k)}$ 
9:    $n_{remove} = \max(1, \min(20, \lceil \rho \cdot |\mathcal{P}^{(k)}| \rceil))$ 
10:   $\mathcal{P}^{remove} = \{i \in \mathcal{P}^{(k)} \mid \phi_i \text{ among top } n_{remove} \text{ values}\}$ 
11:   $\mathcal{P}^{(k+1)} = \mathcal{P}^{(k)} \setminus \mathcal{P}^{remove}$ 
12:   $\mathcal{P}^{removed} = \mathcal{P}^{removed} \cup \mathcal{P}^{remove}$ 
13:   $k = k + 1$ 
14: end while
15: return empty solution and  $\mathcal{P}^{removed} = \mathcal{P}$ 

```

Greedy Recovery Phase: For pods in $\mathcal{P}^{removed}$, we attempt placement using a greedy heuristic that selects the feasible placement with minimum carbon emissions:

$$(j^*, t^*) = \arg \min_{(j,t) \in \mathcal{V}_i \cap \mathcal{A}} e_{ijt} \quad (42)$$

where $\mathcal{A}$ represents the set of currently available node-timeslot pairs.

1.8.4 Complexity Analysis

Multi-Objective Approach: The relaxed MILP has $|\mathcal{P}| \times |\mathcal{F}| \times |\mathcal{T}| + |\mathcal{P}|$ binary variables and $|\mathcal{P}| + |\mathcal{F}| \times |\mathcal{T}| \times 2$ constraints. The theoretical complexity remains NP-hard, but practical solving time may improve due to relaxed constraints.

Iterative Relaxation: In the worst case, the algorithm requires $O(|\mathcal{P}|/\rho)$ iterations, each solving a MILP with decreasing size. The overall complexity is $O(|\mathcal{P}|^2 \cdot T_{solve})$ where T_{solve} is the time to solve a single MILP instance.

Greedy Recovery: The greedy phase has complexity $O(|\mathcal{P}^{removed}| \times |\mathcal{F}| \times |\mathcal{T}|)$, which is polynomial in the problem size.

1.8.5 Implementation Considerations

The relaxation methods are implemented in a hierarchical manner:

1. **Primary approach:** Multi-objective optimization with penalty functions

2. **Fallback 1:** Iterative constraint relaxation when multi-objective fails
3. **Fallback 2:** Greedy placement heuristic for remaining unplaced pods

The penalty weight λ is a critical parameter that balances placement priority against emissions optimization. In practice, values of $\lambda = 10^4$ have been observed to provide reasonable trade-offs, though systematic parameter tuning may be required for different problem instances.

1.9 Implementation Details

We implement the model in PuLP/CBC as follows:

1. Precompute valid placements $\mathcal{V}_i$ (respecting earliest times and deadlines).
2. Create x_{ijt} for $(j, t) \in \mathcal{V}_i$, s_i for all pods, and y_{jt} (Phase 2).
3. **Phase 1:** Solve $\min \sum_i s_i$ subject to assignment and capacity constraints.
4. **Phase 2:** Fix s_i to Phase 1 values; warm-start x_{ijt} ; add activation variables and linking; solve emissions objective.
5. If Phase 2 times out, retry with the dynamic-only fallback objective.
6. Post-process placements to compute per-pod emissions (including allocation of idle+embodied for reporting) and write results.